


# SDEV2171

## Lesson 12: Fetching Remote Data

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

# Lesson Objectives

By the end of this lesson, you should be able to:

- Explain what an API request does in a mobile app
- Explain when a request should run in a screen
- Fetch JSON data when a screen opens
- Store fetched data in state and render it
- Debug common request problems

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Today

| Time | Activity |
|---|---|
| 0:00 | Check your app |
| 0:05 | APIs, JSON, and fetch |
| 0:20 | Code along: request the data, then show it |
| 0:45 | Break |
| 0:55 | Code along: remove the local list, handle a failed request |
| 1:05 | Your turn: a different endpoint |
| 1:20 | Bug swap |
| 1:32 | Checkpoint |

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Check Your App

Run your project from lesson 11 on your development build.

1. Home opens Details, and Go back returns to Home
2. The list shows your study steps
3. Add step and Clear list both work

If any of these fail, fix it now or ask me.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Where the Data Comes From

An **API** is a server your app asks for data. Each URL you can ask is an **endpoint**. The answer usually comes back as **JSON**.

Endpoint: `https://jsonplaceholder.typicode.com/todos?_limit=6`

```
[
  { "userId": 1, "id": 1, "title": "delectus aut autem", "completed": false },
  { "userId": 1, "id": 2, "title": "quis ut nam facilis et officia qui", "completed": false }
]
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

# fetch Takes Time

```
const response = await fetch('https://jsonplaceholder.typicode.com/todos?_limit=6');

if (!response.ok) {
  throw new Error(`Response status: ${response.status}`);
}

const data = await response.json();
```

- `fetch` returns a Promise. `await` waits for it
- A 404 or 500 doesn't throw on its own. Check `response.ok`
- `response.json()` also returns a Promise

WE ARE ESSENTIAL TO ALBERTA | NAIT

# When Should It Run?

Once, when the screen opens.

```
useEffect(() => {
  loadRemoteSteps();
}, []);
```

- `useEffect` runs after the screen renders
- `[]` means: only after the first render

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Step 1: The Request

```
async function loadRemoteSteps() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/todos?_limit=6');

    if (!response.ok) {
      throw new Error(`Response status: ${response.status}`);
    }

    const data = await response.json();
    console.log('first item', data[0]);
  } catch (error) {
    console.error('request failed', error);
  }
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Step 1: The Trigger

```
import { useEffect, useState } from 'react';
```

```
const [remoteSteps, setRemoteSteps] = useState([]);
```

```
useEffect(() => {
  loadRemoteSteps();
}, []);
```

Check: the terminal shows the first item, an object with `userId`, `id`, `title`, and `completed`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Step 2: Store It and Show It

Below the `console.log`:

```
setRemoteSteps(
  data.map((item) => ({
    id: String(item.id),
    label: item.title,
  }))
);
```

In the JSX, use `remoteSteps` instead of `studySteps`:

```
data={remoteSteps}
```

```
{remoteSteps.length === 0 ? (
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Why Map the Data?

| | API item | List item |
|---|---|---|
| Shape | `{ userId, id, title, completed }` | `{ id, label }` |
| `id` | a number | a string, for `keyExtractor` |
| Text | `title` | `label`, which `renderItem` reads |

Store the data in the shape your list already uses.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# The Wrong Trigger

Both of these keep sending requests until you stop them:

```
// Called directly in the component body
loadRemoteSteps();
```

```
// useEffect with no dependency array
useEffect(() => {
  loadRemoteSteps();
});
```

The request sets state. Setting state re-renders the screen. Each render starts another request.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Break

10 minutes

WE ARE ESSENTIAL TO ALBERTA
NAIT

# Step 3: Remove the Local List

Delete:

- the `draftStep` and `studySteps` state
- `handleAddStep` and `handleClearSteps`
- the `TextInput` and the Add step and Clear list buttons
- `TextInput` from the `react-native` import
- the `console.log` in `loadRemoteSteps`

Keep the list, the empty message, and Go back.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# When the Request Fails

Change `todos` to `todoz` in the URL and save.

- The server answers with a 404, so `response.ok` is `false`
- The `throw` jumps to `catch`
- The terminal shows `request failed` and `Response status: 404`
- The screen keeps showing "No study steps yet."

Lesson 13 adds a loading message and an error message. Change the URL back.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Your Turn: A Different Endpoint

Change the request URL to `https://jsonplaceholder.typicode.com/users?_limit=5`

1. Log the first item and find the property with the person’s name
2. Map that property to `label`
3. Check: the list shows five names, starting with Leanne Graham

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Bug Swap

1. You and your partner each get a bug card
2. Make the change on your partner’s laptop while they look away, then save
3. Debug your own app: reproduce, describe, inspect, change one thing, retest
4. When you’re both done, explain your bug to your partner

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Checkpoint

- What endpoint does your screen request?
- When does the request run, and what makes it run only once?
- What does one API item look like, and what does one list item look like?
- What happens on screen when the request fails?

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Before You Leave

1. Your list loads from the API, and Go back works
2. Delete any debugging logs
3. Record your status: **resolved**, **partial**, or **blocked**
4. Exercise: see the end of the lesson README

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Next: Lesson 13

Right now the screen shows "No study steps yet." while it waits, and again if the request fails.

Lesson 13 gives each case its own message:

- **loading:** the request hasn’t finished
- **success:** show the list
- **error:** the request failed

WE ARE ESSENTIAL TO ALBERTA | NAIT